A Rota das Coisas

Luan Rodrigues Araújo[1]

[1]Universidade Estadual de Feira de Santana - UEFS
Av. Transnordestina, S/N, Novo Horizonte, Feira de Santana, Bahia, Brasil, CEP 44.036-900

luanlra002@gmail.com

Abstract. *This work presents the design and implementation of a distributed IoT system using the Go programming language and native UDP and TCP sockets. The architecture is composed of four main components: two sensor nodes (Objeto1 and Objeto2), a central broker, and an actuator. Sensor nodes continuously transmit geolocation data and door status via UDP to the broker, which processes this data, detects critical security conditions, and triggers the actuator via TCP to close doors when necessary. A TCP-based client interface allows human operators to monitor active sensors in real time and manually issue actuation commands. The entire system was containerized using Docker and orchestrated via Docker Compose, enabling scalable simulation of multiple devices. Results demonstrate the viability of the approach in implementing lightweight, responsive, and concurrent IoT communication pipelines using only the Go standard library.*

Resumo. *Este trabalho apresenta o projeto e a implementação de um sistema IoT distribuído utilizando a linguagem de programação Go e sockets nativos UDP e TCP. A arquitetura é composta por quatro componentes principais: dois nós sensores (Objeto1 e Objeto2), um broker central e um atuador. Os nós sensores transmitem continuamente dados de geolocalização e status de porta via UDP ao broker, que processa essas informações, detecta condições críticas de segurança e aciona o atuador via TCP para fechar portas quando necessário. Uma interface cliente baseada em TCP permite que operadores humanos monitorem sensores ativos em tempo real e emitam comandos de atuação manualmente. Todo o sistema foi conteinerizado com Docker e orquestrado via Docker Compose, viabilizando a simulação escalável de múltiplos dispositivos. Os resultados demonstram a viabilidade da abordagem na implementação de pipelines de comunicação IoT leves, responsivos e concorrentes utilizando exclusivamente a biblioteca padrão do Go.*

1. Introdução

A Internet das Coisas (IoT) representa um paradigma tecnológico no qual dispositivos físicos equipados com sensores, atuadores e capacidades de comunicação em rede interagem de forma autônoma para coletar, processar e compartilhar informações com sistemas centralizados ou entre si. Esse paradigma tem impulsionado inovações em domínios que variam de cidades inteligentes e automação industrial a monitoramento ambiental e controle de acesso físico.

No contexto do presente projeto, o desafio proposto consiste no desenvolvimento de um sistema distribuído de monitoramento e controle de objetos inteligentes, em que

sensores IoT reportam periodicamente sua localização geográfica e o estado de uma porta (aberta ou fechada) a um servidor broker. Esse broker, por sua vez, é responsável por consolidar os dados recebidos, detectar situações críticas e acionar automaticamente um componente atuador capaz de intervir no ambiente físico.

A solução foi desenvolvida integralmente na linguagem Go (Golang), fazendo uso exclusivo de sockets nativos dos protocolos UDP e TCP, sem a utilização de frameworks de mensagem ou bibliotecas externas de comunicação. Essa escolha atende às restrições técnicas impostas pelo problema e, ao mesmo tempo, demonstra a capacidade da biblioteca padrão do Go para a construção de sistemas de comunicação de rede robustos e eficientes.

Este relatório contém a justificativa teórica que serviu como base para as decisões de design tomadas, a arquitetura escolhida, a implementação de cada um dos componentes, os testes executados, os resultados obtidos e as limitações encontradas, juntamente com as áreas de melhorias possíveis para o futuro sistema.

2. Fundamentação Teórica

Esta seção detalha os pilares teóricos e as ferramentas tecnológicas que sustentam a solução para o problema de integração de rede e gerenciamento de dispositivos.

2.1. Comunicação em Redes com Sockets UDP e TCP

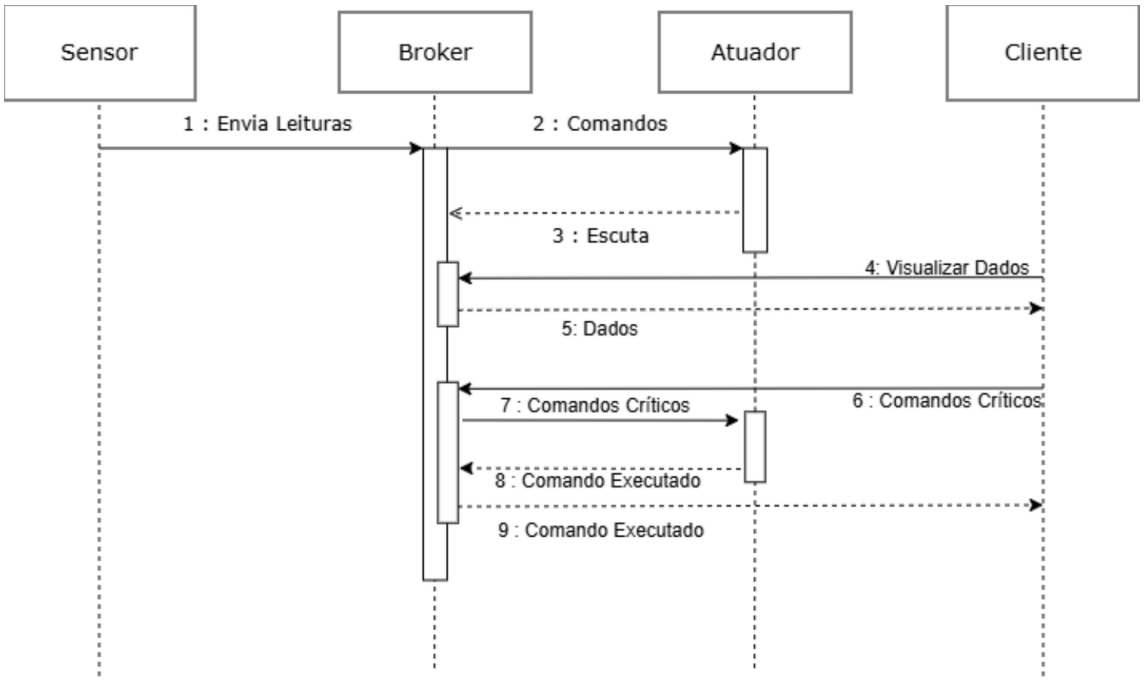

Figure 1. Diagrama de Sequência da Comunicação

A comunicação entre processos distribuídos em redes de computadores pode ser estabelecida por meio de dois protocolos de transporte fundamentais: o TCP (Transmission Control Protocol) e o UDP (User Datagram Protocol). Enquanto o TCP oferece uma conexão orientada, confiável e com garantias de entrega e ordenação de pacotes, o UDP

opera de forma sem conexão e não garante a entrega das mensagens, apresentando, em contrapartida, latência significativamente menor e menor sobrecarga de protocolo.

No presente projeto, a escolha pelo UDP para o canal de comunicação entre os sensores e o broker reflete a natureza dos dados transmitidos: telemetria geoespacial de alta frequência, em que a perda esporádica de um pacote é tolerável e a latência reduzida é prioritária (Tanenbaum and van Steen, 2017a). Já o TCP foi adotado para os canais de controle — comunicação broker-atuador e broker-cliente — onde a confiabilidade e a garantia de entrega das mensagens são críticas, dado que tais mensagens correspondem a comandos de atuação e interações de usuário que não podem ser descartados sem consequências (Ribeiro, 2016).

2.2. Arquitetura IoT com Broker

Em sistemas IoT de médio e grande porte, é comum a adoção de um componente broker, cuja responsabilidade é mediar a comunicação entre múltiplos dispositivos de campo (sensores e atuadores) e os sistemas de controle ou consumo de dados. Esse modelo desacopla os produtores de dados dos consumidores, simplificando o gerenciamento de conexões, a persistência de estado e a aplicação de regras de negócio.

No sistema, o broker desempenha papel central: ele recebe dados de múltiplos sensores via UDP, mantém um mapa de estado atualizado de cada objeto monitorado, aplica lógica de detecção de anomalias e encaminha comandos ao atuador via TCP. Essa arquitetura se alinha ao padrão de publicação-subscrição adaptado para redes de sensores, em que o broker é o único componente com conhecimento global do estado do sistema (Tanenbaum and van Steen, 2017b).

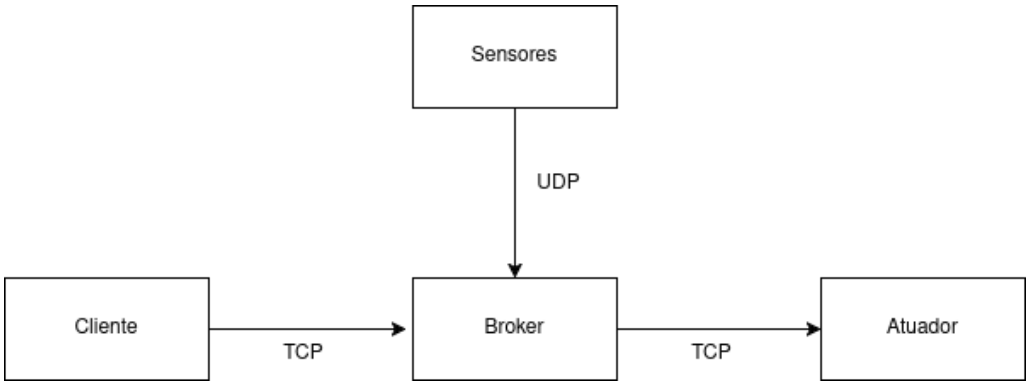

Figure 2. Arquitetura Geral

2.3. Concorrência em Go com Goroutines

A linguagem Go foi projetada com suporte nativo à concorrência por meio de goroutines — unidades de execução leves gerenciadas pelo runtime da linguagem, com pilhas de memória dinamicamente redimensionáveis. Goroutines são muito mais eficientes do que threads do sistema operacional tanto em termos de consumo de memória quanto de custo de criação e troca de contexto, tornando-as ideais para sistemas que gerenciam simultaneamente dezenas ou centenas de conexões de rede.

No broker implementado neste projeto, goroutines são utilizadas de forma extensiva: cada conexão TCP de cliente é atendida em uma goroutine independente, o servidor

UDP opera em uma goroutine dedicada, a rotina de limpeza de sensores inativos é executada em segundo plano em outra goroutine, e os envios ao atuador decorrentes de alertas automáticos também ocorrem em goroutines isoladas para não bloquear o loop principal de recebimento de dados.

2.4. Sincronização com Mutex

Em sistemas concorrentes, o acesso a estruturas de dados compartilhadas por múltiplas goroutines exige mecanismos de sincronização para prevenir condições de corrida. O pacote sync da biblioteca padrão do Go fornece a primitiva sync.Mutex, que garante exclusão mútua no acesso a regiões críticas do código.

No broker, a estrutura MemoriaSensor — que armazena a lista de objetos ativos — é protegida por um sync.Mutex embarcado. Todas as operações de leitura e escrita sobre essa lista, sejam elas provenientes do servidor UDP, das rotinas de limpeza ou dos handlers TCP, realizam as operações de Lock() e Unlock() correspondentes, garantindo a consistência dos dados mesmo sob alta concorrência (Donovan and Kernighan, 2016).

2.5. Contêineres Docker e Docker Compose

O Docker é uma plataforma de virtualização leve que permite empacotar aplicações e suas dependências em contêineres isolados, garantindo reprodutibilidade do ambiente de execução independentemente do sistema operacional hospedeiro. Cada contêiner executa um processo principal e é definido por uma imagem construída a partir de um Dockerfile, que especifica a base do sistema, as dependências, os arquivos da aplicação e o comando de inicialização.

O Docker Compose é uma ferramenta complementar que permite definir e orquestrar múltiplos contêineres a partir de um único arquivo de configuração no formato YAML. No contexto deste projeto, cada componente do sistema (broker, sensores e atuador) possui seu próprio Dockerfile e docker-compose.yaml, possibilitando a instanciação independente de cada serviço e a configuração de redes compartilhadas para comunicação entre contêineres.

3. Metodologia

3.1. Proposta de Solução

A proposta deste projeto consiste no desenvolvimento de um sistema IoT distribuído para o monitoramento em tempo real de objetos inteligentes equipados com sensores de posição geográfica e estado de porta, com capacidade de acionamento automático ou manual de um atuador responsável pelo fechamento das portas. O sistema foi concebido para operar de forma contínua, tolerante a falhas de sensores individuais e escalável para múltiplos dispositivos monitorados simultaneamente.

A solução é composta por quatro entidades principais, cada uma implementada como um programa independente em Go e executada em seu próprio contêiner Docker:

- **Broker (servidor central):** Responsável por receber dados dos sensores, manter o estado global do sistema, detectar anomalias e coordenar a atuação;
- **Objeto1 e Objeto2 (sensores IoT):** Responsáveis por gerar e transmitir periodicamente dados de localização e status de porta ao broker;

- **Atuador:** Responsável por receber comandos do broker e executar a ação de fechamento de porta, retornando confirmação da operação;
- **Cliente:** interface TCP interativa que permite a um operador humano consultar o estado dos sensores ativos e emitir comandos de atuação.

3.2. Arquitetura Geral

A estrutura do sistema é basicamente uma rede com um ponto central, que chamamos de broker. Isso pode ser visualizado de uma forma simplificada. Os dispositivos de sensoriamento se conectam ao broker apenas por meio de mensagens UDP, utilizando a porta 4042. Já o broker mantém duas conexões diferentes via TCP: uma na porta 4041, que é usada pelo operador, e outra na porta 8983, que é usada pelo dispositivo que realiza a ação. É importante notar que os dispositivos de sensoriamento e o dispositivo que realiza a ação não se comunicam diretamente; todas as ações são controladas pelo broker.

Essa separação de responsabilidades e a centralização do estado no broker garantem que o sistema possua uma visão consistente e atualizada do ambiente monitorado a qualquer instante, ao mesmo tempo em que simplifica o design dos componentes de borda (sensores), que se limitam a transmitir dados sem necessidade de manter estado ou lógica de controle complexa.

3.3. Implementação

3.3.1. Estrutura dos Sensores (Objeto1 e Objeto2)

Os componentes Objeto1 e Objeto2 são programas Go estruturalmente idênticos em sua lógica, diferindo apenas nos valores estáticos de identificação (Id e Name). Cada sensor executa um loop infinito no qual, a cada três segundos, gera coordenadas geográficas pseudoaleatórias dentro de um intervalo configurável e serializa um objeto do tipo Object para o formato JSON, transmitindo-o via UDP ao endereço do broker.

A estrutura de dados Object encapsula os seguintes campos: Id (identificador único do dispositivo), Name (nome descritivo), Coordinates (fatia de estruturas Coords contendo Latitude e Longitude como strings formatadas com duas casas decimais), Door (status da porta, fixado como "Aberta") e Time (campo utilizado internamente pelo broker para controle de atividade).

A geração de coordenadas aleatórias é realizada pela função objectData, que recebe os limites mínimo e máximo do intervalo e aplica a fórmula de interpolação linear sobre valores gerados pelo pacote math/rand da biblioteca padrão (The Go Authors, 2026). Os valores resultantes são formatados como strings com duas casas decimais, uma decisão de projeto que viabiliza comparações ordenadas por valor textual no broker para detecção de condições de alerta.

A comunicação UDP é estabelecida por meio da função net.DialUDP, que cria um socket UDP conectado ao endereço de destino. A cada iteração do loop, a mensagem JSON serializada é escrita diretamente no socket, e o programa aguarda três segundos antes da próxima transmissão.

3.3.2. Estrutura do Broker

O broker é o componente mais complexo do sistema, implementando três servidores concorrentes: um servidor UDP para receber dados dos sensores, um servidor TCP para atender clientes operadores, e uma rotina de limpeza periódica de sensores inativos. O ponto de entrada da função main inicializa essas três corrotinas: cleanupSensors e serverUDP são lançadas como goroutines, enquanto serverTCP é executado na goroutine principal.

O estado global compartilhado é mantido na variável currentStatus, do tipo MemoriaSensor, que é uma estrutura composta por um sync.Mutex e uma lista (slice) de objetos do tipo Object. Essa estrutura centraliza todo o estado operacional do sistema e é o único recurso compartilhado entre as goroutines concorrentes.

O servidor UDP, executado pela função serverUDP, aguarda datagramas na porta 4042 utilizando net.ListenUDP. A cada datagrama recebido, o payload é desserializado para um objeto Object via json.Unmarshal. O broker então aplica a lógica de atualização do estado: se o objeto já existe na lista (identificado pelo campo Id), seu registro é atualizado. Caso contrário, um novo registro é inserido. Em ambos os casos, o campo Time é atualizado com o instante atual da recepção, servindo como marca temporal para a detecção de inatividade. Após a atualização, o broker envia uma confirmação textual de volta ao endereço do sensor.

A lógica de detecção de anomalias está implementada no servidor UDP e é aplicada a cada datagrama recebido. A condição de alerta é disparada quando o sensor reporta latitude negativa, longitude negativa e porta aberta simultaneamente, o que representa um objeto em posição geograficamente irregular com porta exposta. Ao detectar essa condição, o broker lança uma goroutine dedicada para acionar o atuador sem bloquear o loop de recebimento de novos dados.

O servidor TCP, executado pela função serverTCP, aguarda conexões na porta 4041 e atende cada cliente em uma goroutine isolada via handleConnectionTCP. Ao receber uma nova conexão, o broker imediatamente serializa e transmite um menu de opções via json.NewEncoder. O cliente pode escolher entre duas opções: (1) listar sensores ativos e visualizar detalhes de um sensor específico, ou (2) acionar o fechamento de porta de um objeto selecionado. O protocolo de comunicação TCP é baseado em troca de mensagens JSON, com tipos Menu (para mensagens do broker) e UserInput (para respostas do cliente).

A rotina cleanupSensors é executada periodicamente a cada sete segundos e remove da lista os sensores que não transmitiram dados dentro da janela de sete segundos anteriores. Essa lógica de expiração garante que a lista de sensores ativos reflita fielmente os dispositivos efetivamente operacionais no momento, evitando que sensores desconectados ou com falha permaneçam indefinidamente na lista.

3.3.3. Estrutura do Atuador

O atuador é um servidor TCP simples que escuta conexões na porta 8983. A cada conexão recebida — proveniente do broker — o atuador desserializa o objeto JSON recebido, que contém os dados do sensor alvo da atuação, e simula a ação de fechamento de porta

imprimindo uma mensagem descritiva no console. Em seguida, serializa e transmite ao broker uma estrutura de confirmação contendo um campo Status com a mensagem de resultado da operação.

O atuador utiliza goroutines para atender múltiplas conexões concorrentes (handleActuator), o que é necessário para suportar cenários em que o broker dispara atuações simultâneas para diferentes objetos. A ausência de estado persistente no atuador simplifica sua implementação e o torna facilmente escalável.

3.3.4. Estrutura do Cliente

O cliente é um programa interativo de linha de comando que estabelece uma conexão TCP com o broker na porta 4041. Após receber e exibir o menu inicial, o usuário digita sua escolha numérica, que é serializada como um objeto UserInput e enviada ao broker. O cliente então aguarda e processa as respostas sequenciais do broker, exibindo listas de sensores, detalhes de objetos ou confirmações de atuação conforme o fluxo da opção selecionada.

O protocolo de comunicação cliente-broker é estritamente orientado a turnos: cliente envia uma mensagem, broker responde, cliente envia novamente se necessário. Essa abordagem síncrona simplifica o tratamento de estado no cliente, dispensando a necessidade de controle de concorrência no componente de interface com o usuário.

3.3.5. Conteinerização e Orquestração com Docker

Cada componente do sistema — broker, Objeto1, Objeto2, atuador e cliente — possui um Dockerfile dedicado que utiliza a imagem oficial golang:1.25-alpine como base. O processo de build compila o binário Go dentro do contêiner de construção e o executa como processo principal (CMD). Essa abordagem garante que os binários sejam compilados com a versão correta do compilador Go independentemente do ambiente hospedeiro (Vitalino and Castro, 2021).

Os arquivos docker-compose.yaml correspondentes definem as configurações de cada serviço, incluindo mapeamentos de porta, variáveis de ambiente (como SERVER_ADDR para configuração do endereço do broker nos sensores), políticas de reinicialização automática e pertencimento à rede virtual iot_net do tipo bridge. A rede bridge do Docker permite que os contêineres se descubram mutuamente pelo nome do contêiner, eliminando a necessidade de endereços IP estáticos na configuração dos sensores (Docker Inc., 2026).

3.4. Experimentos e Resultados

Os experimentos foram conduzidos em um ambiente integralmente conteinerizado com Docker, executado em uma única máquina hospedeira. Cada componente do sistema foi instanciado como um contêiner Docker independente, conectado à rede virtual iot_net do tipo bridge. Essa configuração permitiu simular um ambiente de rede distribuído com múltiplos nós comunicando-se por endereços e portas distintas, reproduzindo fielmente as condições de operação de um sistema IoT real dentro dos limites de um laboratório de desenvolvimento (Vitalino and Castro, 2021).

A utilização de contêineres separados para cada parte do sistema permitiu que várias instâncias de sensores funcionassem juntas sem problemas de recursos. Além disso, foi possível reiniciar automaticamente qualquer parte que falhasse, usando uma configuração específica nos arquivos do Docker Compose (always). Isso também ajudou a manter as dependências organizadas, garantindo que todos os componentes usassem a mesma versão do ambiente de execução Go.

3.5. Simulação de Transmissão de Dados pelos Sensores

Os sensores Objeto1 e Objeto2 foram configurados para transmitir dados a cada três segundos, gerando coordenadas aleatórias dentro do intervalo [-40, 40] para ambos os eixos de latitude e longitude. Durante os testes, observou-se que o broker recebia e processava os datagramas UDP de forma confiável dentro de ambientes de rede local, atualizando o estado de cada sensor na estrutura de dados compartilhada e respondendo ao sensor com uma confirmação textual.

A transmissão periódica em intervalo de três segundos demonstrou-se adequada para manter a presença dos sensores na lista de ativos do broker, uma vez que a janela de expiração foi definida em sete segundos. Sensores que interrompem a transmissão por mais de sete segundos são automaticamente removidos da lista pela rotina cleanupSensors, o que foi verificado ao encerrar um dos contêineres de sensor durante os testes.

3.6. Detecção Automática de Anomalias e Acionamento do Atuador

A lógica de detecção de anomalias implementada no broker foi testada por meio da análise dos dados transmitidos pelos sensores. A condição de alerta é satisfeita quando latitude e longitude são ambas negativas e a porta está aberta. Dado que os sensores geram coordenadas no intervalo simétrico [-40, 40], aproximadamente um quarto das transmissões satisfaz a condição de anomalia (quadrante de latitude negativa e longitude negativa).

Durante os testes, verificou-se que o broker detectou corretamente as condições de alerta e disparou conexões TCP ao atuador para as mensagens correspondentes. O atuador retornou confirmações de atuação que foram exibidas no console do broker. O uso de goroutines para o envio ao atuador garantiu que o loop principal de recebimento UDP não fosse bloqueado durante o processamento das atuações, mantendo a responsividade do sistema para novos dados.

3.7. Monitoramento via Interface Cliente

A interface de cliente TCP foi testada com múltiplas sessões concorrentes, verificando-se que o broker atende cada conexão em uma goroutine independente sem conflitos no acesso à estrutura de dados compartilhada. A listagem de sensores ativos refletiu corretamente o estado atual do sistema em todas as sessões de teste, com adição e remoção de sensores sendo visível em consultas subsequentes.

O mecanismo de acionamento manual via cliente (opção 2 do menu) foi testado com sucesso, confirmando que o broker recebe o índice do sensor selecionado, localiza o objeto correspondente na lista, envia os dados ao atuador e retorna ao cliente a confirmação recebida do atuador.

3.8. Comportamento sob Concorrência

Testes com múltiplos sensores transmitindo dados simultaneamente ao broker validaram a consistência da sincronização implementada com ‘sync.Mutex‘. Nenhuma condição de corrida foi observada durante os testes, e a lista de sensores ativos permaneceu consistente ao longo de toda a execução. A separação entre o loop de recebimento UDP (que escreve na lista) e os handlers TCP de cliente (que leem a lista) foi gerenciada corretamente pelo mecanismo de exclusão mútua, sem degradação de desempenho perceptível nos testes realizados.

4. Conclusão

O projeto demonstrou com sucesso a viabilidade da implementação de um sistema IoT distribuído de monitoramento e controle utilizando exclusivamente a biblioteca padrão da linguagem Go, sem dependências externas de frameworks de comunicação. A arquitetura adotada com separação clara entre sensores UDP, broker central, atuador TCP e cliente de operação mostrou-se robusta e coerente com os princípios de sistemas distribuídos modernos.

A utilização de goroutines para o tratamento concorrente de múltiplas conexões e a proteção do estado compartilhado via sync.Mutex garantiram a confiabilidade do sistema sob condições de concorrência, enquanto a conteinerização com Docker viabilizou a simulação de um ambiente distribuído realista em uma única máquina de desenvolvimento.

Apesar da solidez da implementação, algumas limitações foram identificadas. A comparação baseada na ordem textual de strings para determinar se as coordenadas são negativas em vez do uso de valores numéricos de ponto flutuante, introduz fragilidade na lógica de detecção de anomalias, pois strings como ”-9” seriam corretamente classificadas como negativas, mas strings com diferentes formatações poderiam produzir resultados incorretos. Adicionalmente, o status da porta é fixo como ”Aberta” nos sensores atuais, o que limita a representatividade da simulação em relação a um cenário real.

Como oportunidades de melhoria para versões futuras, destacam-se: a substituição das coordenadas em string por tipos numéricos de ponto flutuante para garantir a corretude das comparações; a implementação de um mecanismo de reconexão automática entre broker e atuador; a adição de autenticação nas conexões TCP para segurança operacional; a exposição de uma API REST ou interface web para visualização do estado do sistema; e a integração com um banco de dados para persistência do histórico de eventos e atuações.

References

Docker Inc. (2026). Docker documentation. Acesso em: 10 abr. 2026.

Donovan, A. A. A. and Kernighan, B. W. (2016). *A linguagem de programação Go*. Novatec Editora, São Paulo, 1 edition.

Ribeiro, F. (2016). Sockets tcp na prática: comunicação cliente-servidor. Embarcados. Acesso em: 10 abr. 2026.

Tanenbaum, A. S. and van Steen, M. (2017a). *Distributed Systems: Principles and Paradigms*. CreateSpace, 3 edition.

Tanenbaum, A. S. and van Steen, M. (2017b). *Distributed Systems: Principles and Paradigms*. CreateSpace, 3 edition.

The Go Authors (2026). The go programming language specification. Acesso em: 10 abr. 2026.

Vitalino, J. F. N. and Castro, M. A. N. (2021). *Descomplicando o Docker*. Novatec Editora, São Paulo.